15. 재귀적 증명 시스템 분석 (Recursive Proof System Analysis)

개요

SP1의 재귀적 증명 시스템은 여러 개의 STARK 증명을 하나의 작은 증명으로 압축하는 핵심 기술입니다. 이 시스템은 대규모 계산에 대한 효율적인 증명을 가능하게 하며, 최종적으로 블록체인에서 검증 가능한 SNARK 증명으로 변환됩니다.

재귀적 구성의 이론적 배경

1. 재귀적 증명의 필요성

대규모 프로그램의 실행은 다음과 같은 문제를 야기합니다:

- **메모리 제약**: 전체 실행 추적을 메모리에 저장하기 어려움
- **증명 크기**: 단일 증명의 크기가 너무 커짐
- **병렬 처리**: 여러 부분을 동시에 증명할 수 없음

재귀적 증명은 이러한 문제를 해결합니다:

```
// 샤드 분할을 통한 병렬 증명 생성
let shards = split_execution_into_shards(execution_trace, shard_size);
let shard_proofs = shards.par_iter()
.map(|shard| prove_shard(shard))
.collect::<Vec<_>>();
```

```
// 재귀적 압축
let compressed_proof = compress_proofs(shard_proofs);
```

2. 증명 합성 (Proof Composition)

SP1은 다음과 같은 단계로 증명을 합성합니다:

단계 1: Core Proof 생성

```
// 각 샤드에 대한 기본 STARK 증명
pub struct CoreProof {
pub shard_proofs: Vec<ShardProof<CoreSC>>,
}

// 샤드 증명 구조
pub struct ShardProof<SC: StarkGenericConfig> {
pub commitment: ShardCommitment<SC>,
pub opened_values: ShardOpenedValues<SC>,
pub opening_proof: OpeningProof<SC>,
pub public_values: Vec<SC::Val>,
}
```

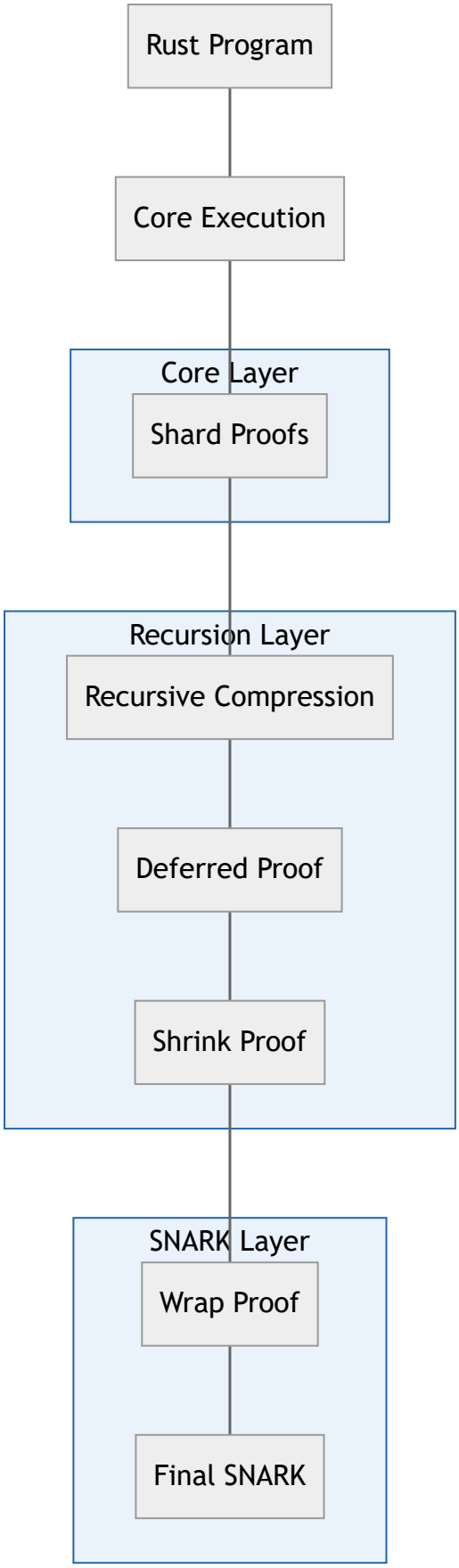
단계 2: Recursive Compression

```
// 재귀적 압축을 위한 머신
```

```
pub  enum RecursionAir<F: PrimeField32 + BinomiallyExtendable<D>, const DEGREE:

MemoryConst(MemoryConstChip<F>),

MemoryVar(MemoryVarChip<F>),

BaseAlu(BaseAluChip),

ExtAlu(ExtAluChip),

Poseidon2Wide(Poseidon2WideChip<DEGREE>),

BatchFRI(BatchFRIChip<DEGREE>),

FriFold(FriFoldChip<DEGREE>),

// ... 기타 칩들

}
```

SP1의 재귀적 증명 파이프라인

1. 전체 파이프라인 구조



2. 각 단계별 상세 분석

Core Proof 단계

```
impl<SC, A> MachineProver<SC, A> for CpuProver<SC, A> {
fn prove(&self, pk: &StarkProvingKey<SC>, records: Vec<A::Record>)
-> Result<MachineProof<SC>, Self::Error> {
// 샤드별 병렬 증명 생성
let shard_proofs = records
.into_par_iter()
.map(|record| {
let traces = self.generate_traces(&record);
let shard_data = self.commit(&record, traces);
self.open(pk, shard_data, &mut challenger.clone())
})
.collect::<Result<Vec<_>, _>>()?;

Ok(MachineProof { shard_proofs })
}
}
```

Recursive Compression 단계

재귀적 압축은 여러 증명을 하나로 합치는 과정입니다:

```
// 압축 머신 구성
pub fn compress_machine<SC: StarkGenericConfig<Val = F>>(config: SC) -> StarkM
let chips = [
```

```
RecursionAir::MemoryConst(MemoryConstChip::default()),
RecursionAir::MemoryVar(MemoryVarChip::default()),
RecursionAir::BaseAlu(BaseAluChip),
RecursionAir::ExtAlu(ExtAluChip),
RecursionAir::Poseidon2Wide(Poseidon2WideChip::<DEGREE>),
RecursionAir::BatchFRI(BatchFRIChip::<DEGREE>),
RecursionAir::Select(SelectChip),
RecursionAir::ExpReverseBitsLen(ExpReverseBitsLenChip::<DEGREE>),
RecursionAir::PublicValues(PublicValuesChip),
]
.map(Chip::new)
.into_iter()
.collect::<Vec<_>>();
StarkMachine::new(config, chips, PROOF_MAX_NUM_PVS, false)
}
```

Deferred Proof 단계

지연된 증명은 추가적인 압축을 제공합니다:

```
// 지연 증명을 위한 특별한 구성
pub  enum SP1ReduceProofWrapper {
Core(SP1ReduceProof<CoreSC>),
Recursive(SP1ReduceProof<InnerSC>),
```

```
}

pub enum SP1RecursionWitnessValues<SC: StarkGenericConfig> {
Core(SP1RecursionWitnessValues<CoreSC>),
Deferred(SP1DeferredWitnessValues<InnerSC>),
Compress(SP1CompressWitnessValues<InnerSC>),
}
```

3. FRI 폴딩 최적화

재귀적 증명에서 FRI 프로토콜의 효율적인 구현:

FRI 폴딩 연산

```
pub struct FriFoldEvent<F> {
pub base_single: FriFoldBaseIo<F>,
pub ext_single: FriFoldExtSingleIo<Block<F>>,
pub ext_vec: FriFoldExtVecIo<Block<F>>,
}

// FRI 폴딩 실행
Instruction::FriFold(instr) => {
let x = memory.mr_unchecked(base_single_addrs.x).val[0];
let z: EF = memory.mr_unchecked(ext_single_addrs.z).val.ext();
```

```
let alpha: EF = memory.mr_unchecked(ext_single_addrs.alpha).val.ext();
for m in 0..ps_at_z.len() {
let p_at_x: EF = mat_opening[m].ext();
let p_at_z: EF = ps_at_z[m].ext();
// 몫 다항식 계산: (p(x) - p(z)) / (x - z)
let quotient = (-p_at_z + p_at_x) / (-z + x);
// 누적값 업데이트
let alpha_pow: EF = memory.mr_unchecked(ext_vec_addrs.alpha_pow_input[m]).val.e
let ro: EF = memory.mr_unchecked(ext_vec_addrs.ro_input[m]).val.ext();
let new_ro = ro + alpha_pow * quotient;
let new_alpha_pow = alpha_pow * alpha;
// 결과 저장
memory.mw_unchecked(ext_vec_addrs.ro_output[m], Block::from(new_ro.as_base_slice
memory.mw_unchecked(ext_vec_addrs.alpha_pow_output[m], Block::from(new_alpha_pow
}
}
```

배치 FRI 처리

```
pub struct BatchFRIEvent<F> {
pub base_vec: BatchFRIBaseVecIo<F>,
pub ext_single: BatchFRIExtSingleIo<Block<F>>,
pub ext_vec: BatchFRIExtVecIo<Block<F>>,
```

```
}

// 배치 FRI 실행
Instruction::BatchFRI(instr) => {
let mut acc = EF::zero();
for m in 0..p_at_zs.len() {
// 배치 처리: Σ αⁱ(p(z) - p(x))
acc += alpha_pows[m] * (p_at_zs[m] - EF::from_base(p_at_xs[m]));
record.batch_fri_events.push(BatchFRIEvent {
base_vec: BatchFRIBaseVecIo { p_at_x: p_at_xs[m] },
ext_single: BatchFRIExtSingleIo { acc: Block::from(acc.as_base_slice()) },
ext_vec: BatchFRIExtVecIo {
p_at_z: Block::from(p_at_zs[m].as_base_slice()),
alpha_pow: Block::from(alpha_pows[m].as_base_slice()),
},
});
}
memory.mw_unchecked(ext_single_addrs.acc, Block::from(acc.as_base_slice()));
}
```

증명 합성 및 검증 과정

1. 공개 값 관리

재귀적 증명에서 공개 값의 일관성 유지:

```
pub struct RecursionPublicValues<T> {
pub committed_value_digest: [T; 32],
pub deferred_proofs_digest: [T; 32],
pub start_pc: T,
pub next_pc: T,
pub start_shard: T,
pub next_shard: T,
pub start_execution_shard: T,
pub next_execution_shard: T,
pub previous_init_addr_bits: [T; 32],
pub last_init_addr_bits: [T; 32],
pub previous_finalize_addr_bits: [T; 32],
pub last_finalize_addr_bits: [T; 32],
pub sp1_version: T,
pub is_complete: T,
}

// 공개 값 커밋
Instruction::CommitPublicValues(instr) => {
let pv_addrs = instr.pv_addrs.as_array();
let pv_values: [F; RECURSIVE_PROOF_NUM_PV_ELTS] =
```

```
array::from_fn(|i| memory.mr_unchecked(pv_addrs[i]).val[0]);

record.public_values = *pv_values.as_slice().borrow();

record.commit_pv_hash_events.push(CommitPublicValuesEvent {

public_values: record.public_values

});

}
```

2. 검증 과정

압축된 증명 검증

```
pub fn verify_compressed(

&self,

proof: &SP1ReduceProof<BabyBearPoseidon2>,

vk: &SP1VerifyingKey,

) -> Result<(), MachineVerificationError<CoreSC>> {

let SP1ReduceProof { vk: compress_vk, proof } = proof;

let mut challenger = self.compress_prover.config().challenger();

// 머신 증명 검증

let machine_proof = MachineProof { shard_proofs: vec![proof.clone()] };

self.compress_prover.machine().verify(compress_vk, &machine_proof, &mut challen

// 공개 값 검증

let public_values: &RecursionPublicValues<_> = proof.public_values.as_slice().b

if !is_recursion_public_values_valid(self.compress_prover.machine().config(), pu
```

```
return Err(MachineVerificationError::InvalidPublicValues(
"Invalid public values in compressed proof".to_string()
));
}
// 검증 키 유효성 확인
if self.vk_verification && !self.recursion_vk_map.contains_key(&compress_vk.has
return Err(MachineVerificationError::InvalidVerificationKey);
}
Ok(())
}
```

래핑된 증명 검증

```
pub fn verify_wrap_bn254(
&self,
proof: &SP1ReduceProof<BabyBearPoseidon2Outer>,
vk: &SP1VerifyingKey,
) -> Result<(), MachineVerificationError<OuterSC>> {
let mut challenger = self.wrap_prover.config().challenger();
let machine_proof = MachineProof { shard_proofs: vec![proof.proof.clone()] };
let wrap_vk = self.wrap_vk.get().expect("Wrap verifier key not set");
self.wrap_prover.machine().verify(wrap_vk, &machine_proof, &mut challenger)?;
Ok(())
```

```
}
```

성능 최적화 전략

1. 메모리 관리

재귀적 증명에서의 효율적인 메모리 사용:

```
pub struct MemVec<F> {
// 메모리 벡터 구현
// 효율적인 주소 매핑과 캐싱
}

// 메모리 접근 최적화
impl<F> MemVec<F> {
pub fn mr_unchecked(&self, addr: Address<F>) -> &MemoryEntry<F> {
// 안전하지 않은 빠른 메모리 읽기
}
pub fn mw_unchecked(&mut self, addr: Address<F>, val: Block<F>) {
// 안전하지 않은 빠른 메모리 쓰기
}
}
```

2. 병렬 처리

```
// 병렬 실행 블록
SeqBlock::Parallel(vec) => {
state.record.append(
&mut vec
.par_iter()
.map(|subprogram| {
// 병렬 실행 - witness stream 없이
Self::execute_raw(env, subprogram, root_program, None)
})
.try_reduce(fresh_record, |mut record, mut res| {
record.append(&mut res);
Ok(record)
})?
);
}
```

3. 형태 최적화 (Shape Optimization)

```
pub enum SP1ProofShape {
Recursion(OrderedShape),
Compress(Vec<OrderedShape>),
Deferred(OrderedShape),
Shrink(OrderedShape),
```

```
}

// 압축 프로그램 형태
pub enum SP1CompressProgramShape {
Recursion(SP1RecursionShape),
Compress(SP1CompressWithVkeyShape),
Deferred(SP1DeferredShape),
Shrink(SP1CompressWithVkeyShape),
}

// 형태별 최적화된 높이 설정
pub fn shrink_shape() -> RecursionShape {
let shape = HashMap::from([
(Self::MemoryVar(MemoryVarChip::default()), 18),
(Self::Select(SelectChip), 18),
(Self::MemoryConst(MemoryConstChip::default()), 17),
(Self::BatchFRI(BatchFRIChip::<DEGREE>), 17),
(Self::BaseAlu(BaseAluChip), 17),
(Self::ExtAlu(ExtAluChip), 18),
(Self::ExpReverseBitsLen(ExpReverseBitsLenChip::<DEGREE>), 17),
(Self::Poseidon2Wide(Poseidon2WideChip::<DEGREE>), 16),
(Self::PublicValues(PublicValuesChip), PUB_VALUES_LOG_HEIGHT),
].map(|(chip, log_height)| (chip.name(), log_height)));
```

```
RecursionShape { inner: shape }

}
```

실제 사용 예제

1. 기본 재귀적 증명 생성

```
// 피보나치 수열 계산의 재귀적 증명

pub fn fibonacci_recursive_proof() {

let n = 10;

let instructions = once(instr::mem(MemAccessKind::Write, 1, 0, 0))

.chain(once(instr::mem(MemAccessKind::Write, 2, 1, 1)))

.chain((2..=n).map(|i| instr::base_alu(BaseAluOpcode::AddF, 2, i, i - 2, i - 1))

.chain(once(instr::mem(MemAccessKind::Read, 1, n - 1, 34)))

.chain(once(instr::mem(MemAccessKind::Read, 2, n, 55)))

.collect::<Vec<_>>();

let program = linear_program(instructions).unwrap();

// 재귀적 머신에서 실행

let machine = RecursionAir::machine_wide_with_all_chips(BabyBearPoseidon2::defa

let (pk, vk) = machine.setup(&program);

// 증명 생성 및 검증

let mut runtime = Runtime::new(Arc::new(program), SC::new().perm);

runtime.run().unwrap();
```

```
run_test_machine(vec![runtime.record], machine, pk, vk)
.expect("Verification failed");
}
```

2. 복잡한 암호학적 연산

```
// 확장체 노름 계산의 재귀적 증명
pub fn field_norm_recursive_proof() {
let mut instructions = Vec::new();
let mut rng = StdRng::seed_from_u64(0xDEADBEEF);
let mut addr = 0;
for _ in 0..100 {
let inner: [F; 4] = std::iter::repeat_with(|| {
core::array::from_fn(|_| rng.sample(rand::distributions::Standard))
})
.find(|xs| !xs.iter().all(F::is_zero))
.unwrap();
let x = BinomialExtensionField::<F, D>::from_base_slice(&inner);
let gal = x.galois_group();
let mut acc = BinomialExtensionField::one();
instructions.push(instr::mem_ext(MemAccessKind::Write, 1, addr, acc));
for conj in gal {
instructions.push(instr::mem_ext(MemAccessKind::Write, 1, addr + 1, conj));
```

```
instructions.push(instr::ext_alu(ExtAluOpcode::MulE, 1, addr + 2, addr, addr + 1

addr += 2;

acc *= conj;

}

let  base_cmp: F = acc.as_base_slice()[0];

instructions.push(instr::mem_single(MemAccessKind::Read, 1, addr, base_cmp));

addr += 1;

}

test_recursion_linear_program(instructions);

}
```

결론

SP1의 재귀적 증명 시스템은 다음과 같은 핵심 특징을 제공합니다:

1. **확장성**: 임의 크기의 프로그램을 효율적으로 증명
2. **모듈성**: 각 단계가 독립적으로 최적화 가능
3. **병렬성**: 샤드별 병렬 처리를 통한 성능 향상
4. **호환성**: 다양한 SNARK 백엔드와의 연동

이러한 재귀적 구조를 통해 SP1은 실용적인 영지식 가상 머신을 구현하며, 블록체인과 같은 제약된 환경에서도 복잡한 계산의 증명을 가능하게 합니다.